ChessPrep AI — Documento Técnico de Diseño (Final)

Fecha: 30 de julio de 2026

---

1. Resumen del proyecto

ChessPrep AI es un asistente de preparación ajedrecística que, dado el nombre de un rival, localiza sus partidas públicas (online y/o presenciales), las analiza estadísticamente y con motor de ajedrez, y genera un informe redactado que distingue explícitamente entre datos objetivos, estadísticas, inferencias e hipótesis.

Principios de diseño que condicionan toda la arquitectura:

- **Nunca inventar datos.** Todo dato mostrado debe ser trazable a una fuente y a un conjunto de partidas concreto.
- **Multi-fuente por diseño.** Ninguna fuente externa es obligatoria ni fiable al 100%; el sistema degrada con elegancia si una fuente falla o no existe.
- **Módulos desacoplados.** Cada uno de los 17 módulos originales debe poder desarrollarse, probarse y desplegarse de forma independiente.
- **Confianza cuantificada.** Cualquier conclusión "blanda" (fortalezas, debilidades, cambios de repertorio) va acompañada de un nivel de confianza calculado, no asignado a mano.
- **Separación online / OTB / combinado.** Todo análisis debe poder acotarse explícitamente por tipo de partida.
- **El LLM nunca calcula, solo redacta.** El Informe IA (Módulo 11) recibe datos ya validados por el Motor de Análisis y el motor de ajedrez; nunca ve PGN crudo ni infiere estadísticas por su cuenta.

---

2. Stack tecnológico

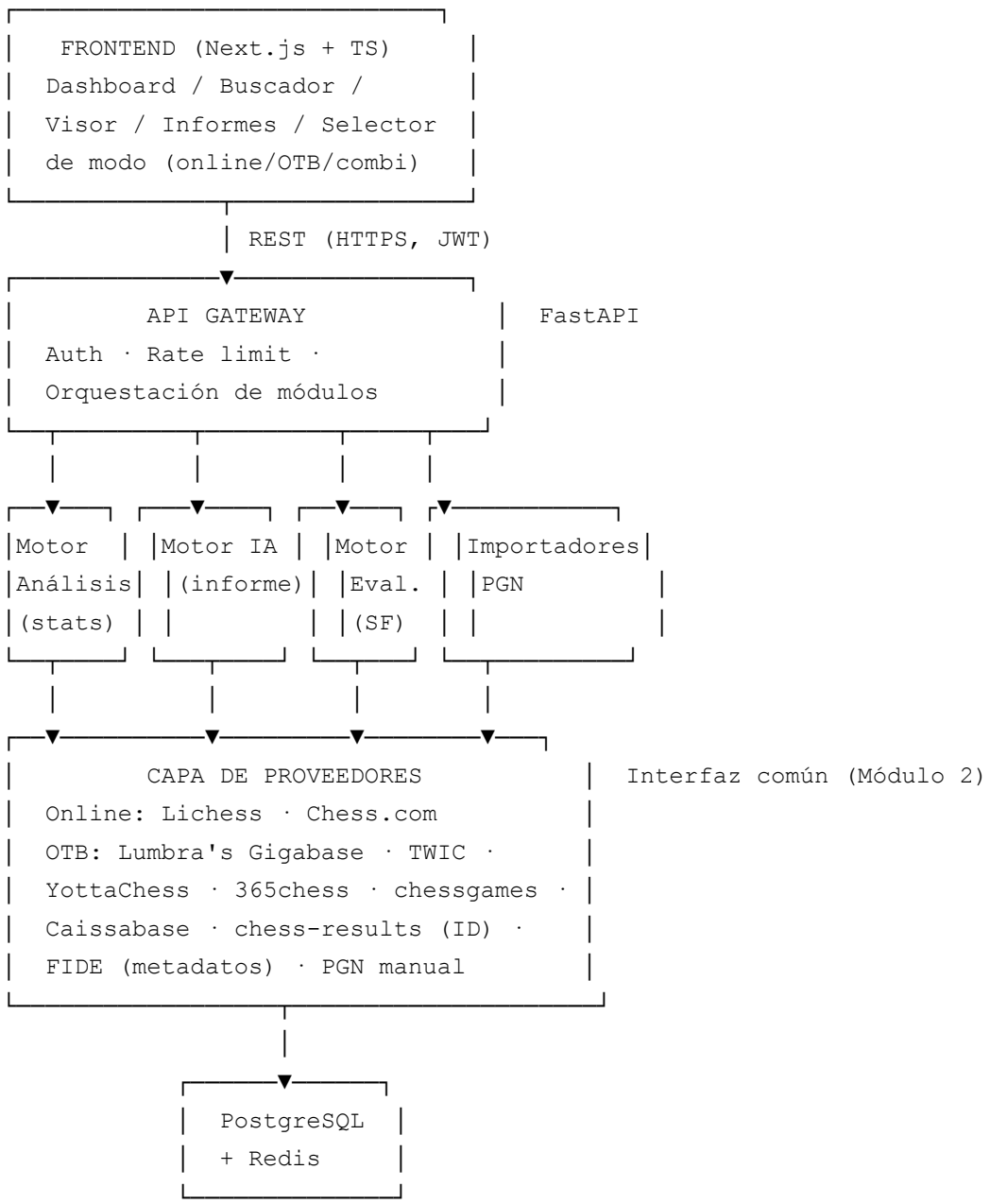
Capa	Elección	Motivo

Frontend web	**Next.js (React) + TypeScript + Tailwind CSS**	SSR/SSG para el dashboard, rutas dinámicas por jugador, modo oscuro fácil, ecosistema maduro para tableros interactivos (react-chessboard, chess.js)
App multiplataforma	**PWA (base) + Capacitor** para empaquetar el mismo código como app nativa iOS/Android	Resuelve la distribución a amigos en móvil y escritorio sin duplicar frontend (ver sección 9)
Backend API	**FastAPI (Python)**	El núcleo de valor (Módulos 4–9, 11, 13) es análisis estadístico y de motor sobre PGN, no CRUD tradicional; Python tiene el ecosistema de ajedrez más maduro (`python-chess`) y facilita la integración con Stockfish
Motor de ajedrez / parsing PGN	**python-chess**	Parseo de PGN, generación de FEN, detección de aperturas por ECO, comunicación UCI con Stockfish
Motor de evaluación	**Stockfish** (self-hosted, GPLv3, gratuito) vía protocolo UCI, orquestado desde workers	Analiza cada partida jugada por el rival para detectar errores reales, no solo estadística de resultados (ver sección 7)
Base de datos relacional	**PostgreSQL**	Modelo claramente relacional (jugadores, partidas, torneos, aperturas, evaluaciones); JSONB para campos semi-estructurados de proveedores
Cache / colas	**Redis**	Cache de resultados de proveedores externos, cola de trabajos para ingesta y análisis de motor
Motor IA (informes)	**API de Claude**	Redacta datos ya calculados; nunca calcula estadísticas
Colas de trabajo	**Celery / RQ**	Ingesta, análisis estadístico y análisis de motor son todos asíncronos y potencialmente pesados
Búsqueda de jugadores	**PostgreSQL full-text / trigram (pg_trgm)**	Suficiente para nombre/apellidos/país sin motor de búsqueda dedicado en la v1

Infraestructura	**Docker Compose** en desarrollo; contenedores independientes por servicio en producción	Frontend, backend, BD, motor de análisis, motor de evaluación e importadores completamente separados

---

3. Arquitectura general



Cada bloque se despliega como servicio independiente; se comunican por API interna, nunca importando código directamente entre capas.

---

4. Capa de proveedores (Módulo 2) — patrón central

Interfaz común que todo proveedor debe implementar:

```
class GameProvider(Protocol):
    name: str
    game_type: Literal["online", "otb"]           # fijo por proveedor
    reliability: Literal["official", "community", "user_upload"]

    def search_player(self, query: PlayerQuery) -> list[PlayerCandidate]: ...
    def fetch_games(self, player_id: PlayerRef, since: date | None) -> list[RawGa
    def is_available(self) -> bool:  # health-check, no asumir API siempre viva
        ...
```

Modelo común de partida (`RawGame`), normalizado independientemente del proveedor:

```
class RawGame(BaseModel):
    source: str                  # "lichess" | "chesscom" | "twic" | "lumbrasgiga
    game_type: Literal["online", "otb"]
    source_confidence: Literal["official", "community", "user_upload"]
    pgn: str
    white: PlayerRef
    black: PlayerRef
    result: Literal["1-0", "0-1", "1/2-1/2", "*"]
    date: date | None
    event: str | None
    eco: str | None
    time_control: str | None
    fetched_at: datetime
```

Deduplicación: hash normalizado sobre (jugadores, fecha, resultado, primeros N movimientos) — no depende de IDs de proveedor, ya que no todos los tienen.

4.1 Proveedores — lista completa

Online:

Proveedor	Tipo de acceso	Notas
Lichess API	API pública, requiere token	Export por usuario, filtros por fecha

Chess.com API	API pública, sin auth	Archivos mensuales por usuario

OTB:

Proveedor	Tipo de acceso	Notas
Lumbra's Gigabase	Descarga de dump (PGN/SCID), gratuito	**Fuente OTB principal recomendada**: viene ya separada estrictamente en OTB (~10,3M partidas) y online curado, con deduplicación activa. Ideal como carga bulk periódica
TWIC	Boletines PGN semanales, gratuito	Complemento de actualidad para torneos muy recientes
YottaChess (YottaBase)	Web buscable por nombre, gratuito	Fallback de búsqueda por nombre
365chess.com	Web buscable por nombre, gratis con límites de descarga	Fallback de búsqueda por nombre
chessgames.com	Web buscable por nombre, gratis	Mejor cobertura de jugadores conocidos/históricos
Caissabase	Descarga de dump único, gratuito	Complemento de bulk
chess-results.com	Web (crosstables/pairings), gratis	No da PGN — sirve solo para **confirmar identidad** (ID FIDE, federación) antes de buscar partidas en otra fuente
FIDE (ratings.fide.com)	Web, gratis	Solo metadatos de perfil (Elo, título, federación), no partidas
Importación manual PGN	N/A	Red de seguridad, siempre disponible

Importante: ninguno de los proveedores OTB web (365chess, chessgames, YottaChess) tiene API pública documentada — la ingesta sería scraping HTML. Hay que revisar sus términos de uso, aplicar rate-limiting agresivo con user-agent identificable, y tratarlos como **fallback de baja frecuencia**, no como sincronización masiva constante.

Orden de prioridad de ingesta: Lumbra's Gigabase (bulk OTB) → TWIC (actualidad) → Lichess/Chess.com (online) → 365chess/chessgames/YottaChess (fallback por nombre) →

importación manual PGN.

---

5. Modelo de datos (entidades principales)

- **Player**: id interno, fide_id (nullable), nombre, apellidos, federación, título, fotos, elos (std/rapid/blitz) con histórico.
- **PlayerAlias**: nombres alternativos/variantes detectadas en distintas fuentes (para no fusionar perfiles automáticamente, Módulo 1).
- **Game**: normalizado desde `RawGame`, con FK a `Player` (blancas/negras), `Tournament`, `Opening`, y campo `game_type` ("online"/"otb").
- **Tournament**: nombre, fechas, formato, fuente.
- **Opening**: código ECO, nombre, variante — catálogo propio derivado de python-chess/ECO estándar.
- **EngineEvaluation**: evaluación de motor por jugada (ver sección 7).
- **AnalysisReport**: snapshot inmutable por (jugador, modo de análisis, fecha de generación, nº de partidas, fuentes usadas) — no se recalcula en caliente, se versiona.
- **ConfidenceScore**: documenta cómo se calculó cada nivel de confianza (nº de partidas, dispersión temporal, tamaño de muestra por modo, etc.) — trazabilidad exigida desde el inicio.

---

6. Modos de análisis: online / OTB / combinado

No son pipelines distintos — es un **filtro** sobre datos ya etiquetados en el momento de la ingesta.

- Cada `GameProvider` declara de forma estática su `game_type` (online u OTB), sin heurísticas de clasificación posteriores.
- La API expone el modo como parámetro explícito:

```
GET /players/{id}/stats?mode=online|otb|combined
GET /players/{id}/openings?mode=online|otb|combined&color=white|black
GET /players/{id}/report?mode=online|otb|combined
```

- Si no se especifica, el backend devuelve **combinado**, pero marca explícitamente en la

respuesta cuántas partidas de cada tipo lo componen — nunca se presenta un número sin decir de qué mezcla sale.

- El prompt del Informe IA incluye el modo como dato estructurado obligatorio, para que el informe lo declare: *"Se analizaron 92 partidas OTB (fuente principal: Lumbra's Gigabase, TWIC) entre 2021 y 2026. No se han incluido partidas online en este informe."* Esto evita sesgar fortalezas/debilidades mezclando estilos de juego distintos (más arriesgado online, más cuidadoso en torneo) sin avisar.
- En la UI: selector de 3 estados (Online / OTB / Combinado) en la ficha de jugador, visor de aperturas e informe, sin re-arquitectura de frontend.

---

7. Motor de evaluación (Stockfish) — análisis por partida y en agregado

Los Módulos 8–9 (fortalezas/debilidades) dejan de basarse solo en resultados y frecuencia de apertura: con Stockfish se sabe qué movimientos concretos fueron errores.

Motor elegido: Stockfish, gratuito y de código abierto (GPLv3), autoalojado vía protocolo UCI. No hay motivo para pagar alternativas comerciales: es el motor más fuerte del mundo actualmente y su licencia permite distribución libre (incluyendo el código fuente).

7.1 Análisis por partida individual

1. Se parsea el PGN a la secuencia de posiciones (FEN por jugada) con `python-chess`.
2. Un worker asíncrono manda cada posición a Stockfish (profundidad configurable, p. ej. 16-18) y recoge la evaluación en centipawns.
3. Se calcula la pérdida de evaluación (`centipawn loss`) de cada jugada respecto a la mejor jugada del motor.
4. Clasificación automática con umbrales estándar:

```
class MoveClassification(str, Enum):
    BEST = "best"
    GOOD = "good"
    INACCURACY = "inaccuracy"   # cp_loss > 50
    MISTAKE = "mistake"          # cp_loss > 100
    BLUNDER = "blunder"          # cp_loss > 300
```

5. Registro `EngineEvaluation` por jugada:

```
class EngineEvaluation(BaseModel):
    game_id: UUID
    ply_number: int
    fen_before: str
    move_played: str
    best_move: str
    eval_before: int        # centipawns
    eval_after: int
    centipawn_loss: int
    classification: MoveClassification
    game_phase: Literal["opening", "middlegame", "endgame"]
```

6. Visible en el **Visor de partidas (Módulo 10)**: gráfico de evaluación jugada a jugada, con blunders/errores marcados sobre el tablero.

7.2 Análisis agregado — lo que alimenta fortalezas y debilidades

- **Debilidades objetivas por fase de partida:** ¿comete más blunders en el medio juego o en el final?
- **Debilidades por estructura/apertura:** cruzando `EngineEvaluation` con `Opening` /ECO, se puede afirmar (con partidas suficientes) que el rival juega objetivamente peor en cierto tipo de estructura, no solo que "gana menos" ahí.
- **Fortalezas objetivas:** aperturas/estructuras con centipawn loss consistentemente bajo, no solo buen resultado (que podría deberse a rivales más débiles).
- **Momentos críticos recurrentes:** patrones como "blunder recurrente al salir de la apertura, jugadas 12-15", agregando por número de jugada.

Esta capa se combina con `ConfidenceScore` : una debilidad detectada por el motor con pocas partidas de muestra se muestra con confianza baja, igual que las inferencias puramente estadísticas.

7.3 Coste computacional y decisiones de diseño

- **Análisis progresivo, no bloqueante:** el informe estadístico se muestra al instante; el análisis de motor se procesa en background con indicador de progreso ("120/243 partidas analizadas con motor").
- **Profundidad adaptativa:** profundidad baja por defecto para todas las partidas; "análisis profundo" bajo demanda para partidas concretas (para esto, una API externa gratuita tipo Chess-API.com es razonable puntualmente; para el bulk en background, Stockfish self-hosted compensa mejor).
- **No re-analizar:** cachear `EngineEvaluation` por partida (hash de la partida).

- **Escalado horizontal:** varios workers en paralelo, cada uno con su propia instancia de Stockfish — trivialmente paralelizable porque cada partida es independiente.

---

8. Mapeo módulos originales → componentes técnicos

Módulo original	Componente técnico
1. Buscador de jugadores	Endpoint `/players/search` + `PlayerAlias` + capa de proveedores (metadatos)
2. Agregador de partidas	Capa de proveedores + cola de ingesta asíncrona
3. Perfil del jugador	Vista agregada sobre `Player` + `Game` (materializada, refrescada por job)
4–7. Estadísticas / aperturas / repertorio / rendimiento	Motor de Análisis (Python, pandas), filtrable por `mode`
8–9. Fortalezas / debilidades	Motor de Análisis + Motor de Evaluación (Stockfish) + `ConfidenceScore`
10. Visor de partidas	React + `chess.js` + `react-chessboard`, con overlay de `EngineEvaluation`
11. Informe IA	Recibe solo datos ya calculados (estadística + motor), nunca PGN crudo
12–13. Comparadores	Reutiliza el Motor de Análisis sobre dos `Player` en paralelo
14. Importación PGN	Proveedor `pgn_upload`, mismo pipeline de normalización
15. Perfil propio	Mismo modelo `Player`, marcado `is_self=true`, permisos de visibilidad propios
16. Calculadoras Elo	Módulo stateless independiente, sin dependencia de BD
17. Dashboard	Vistas agregadas cacheadas en Redis

---

9. API — endpoints principales (borrador)

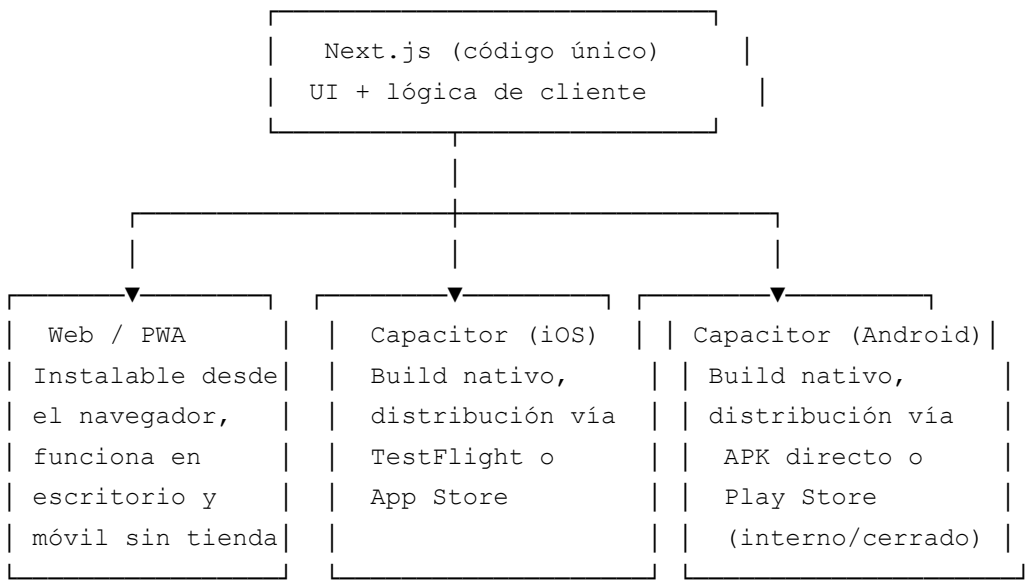
```
GET  /players/search?q=...
GET  /players/{id}
POST /players/{id}/refresh-games              # dispara ingesta asíncrona
GET  /players/{id}/games?mode=online|otb|combined
GET  /players/{id}/stats?mode=online|otb|combined
GET  /players/{id}/openings?mode=...&color=white|black
GET  /players/{id}/report?mode=online|otb|combined
POST /players/{id}/report/generate
GET  /games/{id}/engine-analysis                # jugada a jugada
POST /pgn/import
GET  /compare?a={id1}&b={id2}
GET  /elo/calculators/...
```

---

10. Estrategia multiplataforma — móvil (iOS/Android) y escritorio

Objetivo: un amigo se lo pueda **descargar e instalar** en su móvil (iOS y Android) y en su ordenador, con una sola base de código.

10.1 Enfoque: PWA + Capacitor sobre el mismo Next.js



Capacitor empaqueta la misma web app en un contenedor nativo (WebView), reutilizando el 100% del código Next.js — no hay que reescribir el frontend. Si más adelante se quiere

presencia real en App Store/Play Store, el mismo proyecto se sube sin duplicar trabajo.

10.2 Distribución práctica a amigos (sin revisión de tiendas)

Plataforma	Vía más rápida para un grupo cerrado
Android	APK firmado desde Capacitor, compartido directamente (o "internal testing" en Play Console, instalable por enlace)
iOS	**TestFlight** — hasta 10.000 testers externos sin revisión completa de App Store, solo un enlace de invitación. Requiere cuenta de desarrollador Apple (99$/año)
Escritorio	La PWA instalable desde Chrome/Edge cubre esto sin empaquetado adicional. Un `.exe` / `.dmg` "de verdad" se puede añadir más adelante con **Tauri**

Recomendación para el primer lanzamiento: PWA instalable (Android + escritorio, cero fricción) + TestFlight para iOS.

10.3 Impacto en el backend

Ninguno relevante: FastAPI ya es una API REST consumida igual por web, PWA instalada o build de Capacitor. Añadir:

- **Auth por token (JWT)** en vez de cookies de sesión, para funcionar igual dentro del WebView.
- **CORS** configurado explícitamente para varios orígenes cliente.

---

11. Escalabilidad futura

- **Detección de blunders avanzada / planes de apertura:** ya cubierto en gran parte por el Motor de Evaluación (sección 7); ampliable a sugerencias de repertorio basadas en huecos detectados en el rival.
- **Preparación específica para torneo:** filtro adicional sobre el Motor de Análisis (rango de fechas + lista de rivales del torneo).
- **Exportación PDF:** capa de renderizado sobre `AnalysisReport` (ya versionado e inmutable).
- **Sincronización con bases de datos personales / entrenamiento por rivales frecuentes:** reutiliza el modelo `Player` con `is_self=true` y el histórico de informes ya

generados.

---

12. Próximos pasos

1. Añadir `game_type` al esquema de BD desde el principio (evita migración posterior).
2. Prototipar ingesta con **Lumbra's Gigabase** (OTB) + **Lichess** (online) — validar el modelo `Game` con ambos modos antes de sumar el resto de fuentes.
3. Levantar Stockfish self-hosted en un contenedor de prueba y validar el pipeline Stockfish → `EngineEvaluation` sobre pocas partidas antes de conectarlo al flujo completo.
4. Definir y validar manualmente los umbrales de `MoveClassification` (inaccuracy/mistake/blunder) contra partidas conocidas.
5. Construir el Motor de Análisis estadístico sobre los datos ya ingeridos.
6. Conectar el Informe IA (Módulo 11) al final, cuando ya existan estadísticas y evaluaciones reales que redactar.
7. Configurar Capacitor sobre el proyecto Next.js en cuanto haya una primera versión navegable del dashboard.
8. Crear cuenta de Apple Developer si se quiere TestFlight (la aprobación puede tardar días).

Documento final consolidado — sustituye a las versiones anteriores del diseño técnico.